

数理工学モデル化演習 第 9 回レポート

今村隼人

2026-06-13

1. 設定

課題 1 と課題 2 では、ページ 24 と同じく $\mu = 1$, $E = \frac{\lambda}{\mu} = \lambda$, $c = 10$ とした。課題 3 では $\lambda = 8$, $\mu = 1$ とし、 $c = 10, 15, 20$ の分布を比較した。

2. 結果

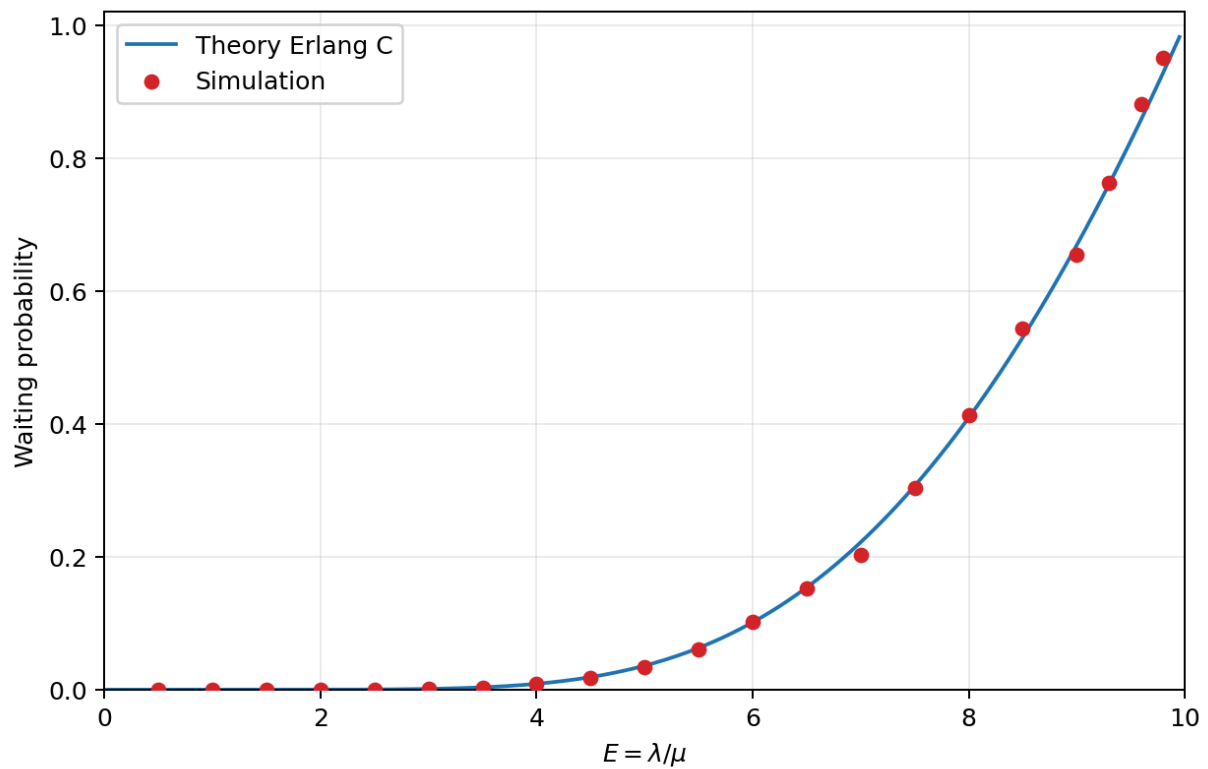


Figure 1: 課題 1 : $c = 10$ の待つ確率

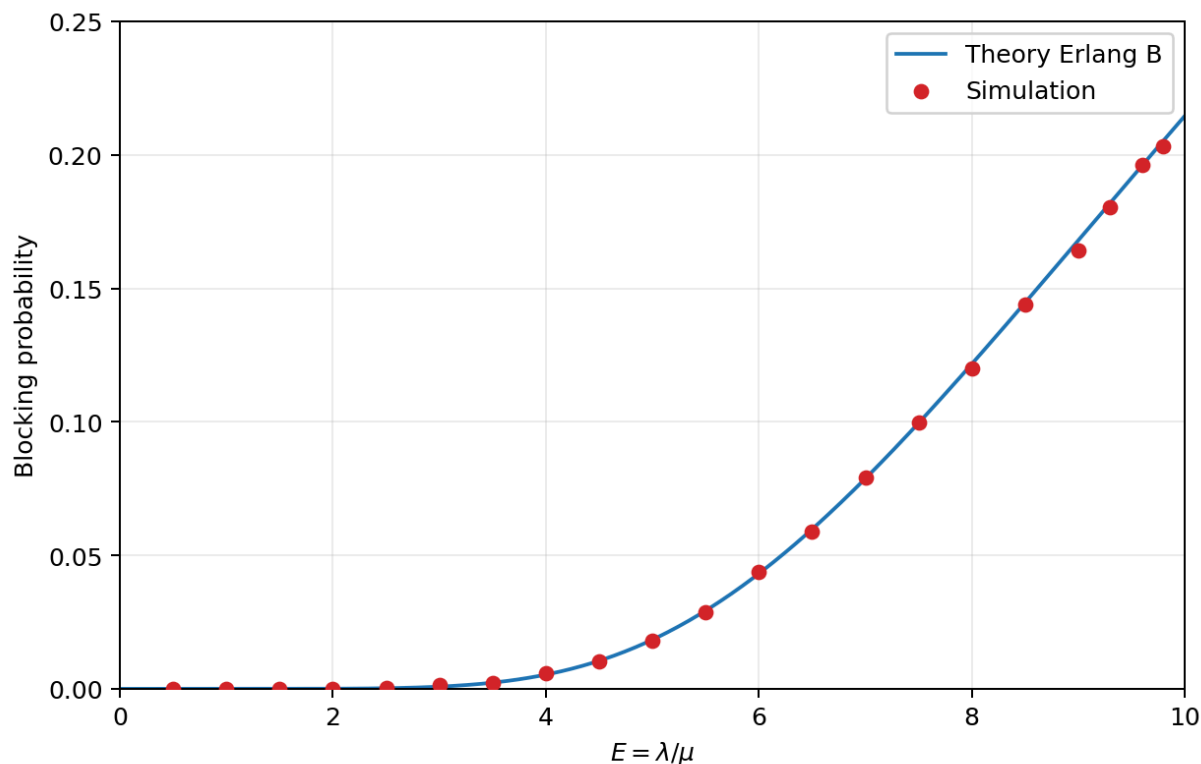


Figure 2: 課題 2 : $c = 10$ のブロッキング確率

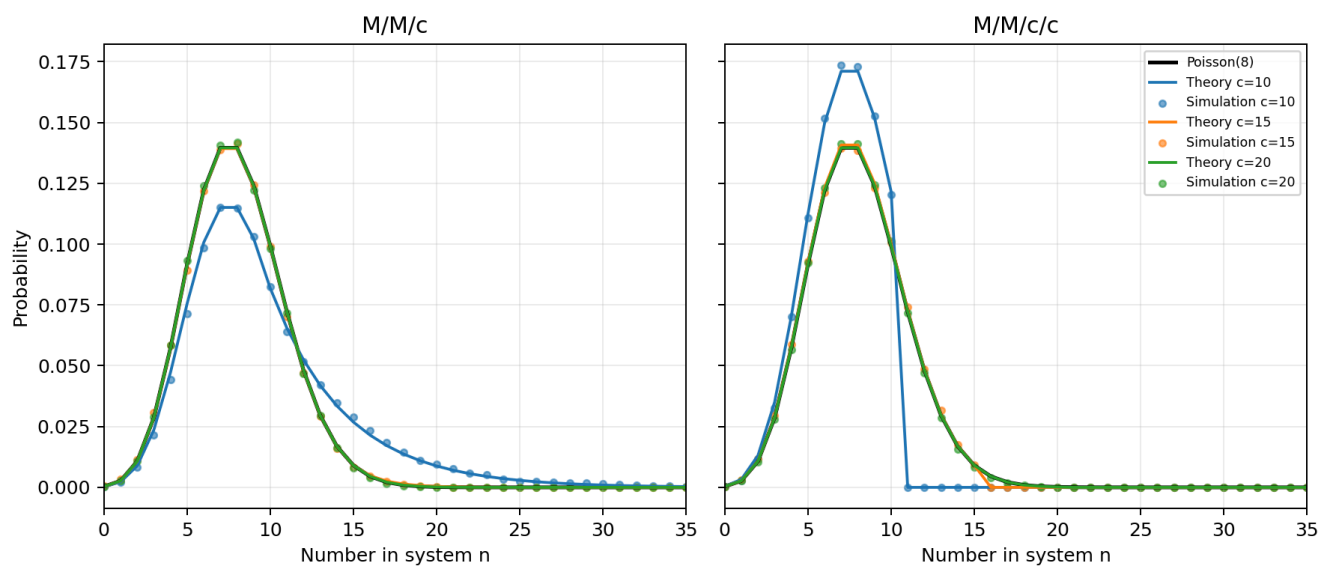


Figure 3: 課題 3 : c を大きくしたときの系内客数分布

3. 考察

課題1の待つ確率は、 E が小さい範囲ではほぼ0だが、 E がサーバ数 $c = 10$ に近づくと急に1へ近づいた。シミュレーション点はErlang Cの理論曲線とほぼ一致し、イベントシミュレーションで待ち確率を再現できている。課題2のブロッキング確率も E とともに増加するが、同じ E では待つ確率よりかなり小さい。これはM/M/c/cモデルでは満員時の到着客を棄却するため、待ち行列を作るM/M/cモデルより系内客数が増えにくいからである。課題3では c を大きくするとM/M/cとM/M/c/cの分布はいずれもPoisson(8)に近づき、十分にサーバ数が多いと容量制限や待ち行列の影響が小さくなることが分かる。

4. ソースコード

```
from __future__ import annotations

import math
import random
from collections import defaultdict
from pathlib import Path

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np

OUT_DIR = Path(__file__).resolve().parent
MU = 1.0
C_TASK = 10
EVENTS = 180_000
WARMUP = 30_000

def exp(rate: float, rng: random.Random) -> float:
    return rng.expovariate(rate)

def simulation_mmcc(
    lam: float,
    mu: float,
    c: int,
    events: int = EVENTS,
    warmup: int = WARMUP,
    seed: int = 0,
) -> tuple[float, dict[int, float]]:
    j = 0
    block = 0
    arrivals = 0
    time_include = 0.0
    occupancy_time: dict[int, float] = defaultdict(float)
    rng = random.Random(seed)

    for state in range(events):
        j_before = j
        if j == 0:
            arrival_interval = exp(lam, rng)
            service_interval = math.inf
        else:
            arrival_interval = exp(lam, rng)
            service_interval = exp(min(j, c) * mu, rng)

        is_arrival = arrival_interval <= service_interval
        interval = arrival_interval if is_arrival else service_interval

        if state >= warmup:
            time_include += interval
            occupancy_time[j_before] += interval

        if is_arrival:
            if state >= warmup:
```

```

        arrivals += 1
        if j_before >= c:
            block += 1
        if j_before < c:
            j += 1
    else:
        j -= 1

distribution = {
    n: value / time_include for n, value in sorted(occupancy_time.items())
}
block_rate = block / arrivals
return block_rate, distribution

def simulation_mmc(
    lam: float,
    mu: float,
    c: int,
    events: int = EVENTS,
    warmup: int = WARMUP,
    seed: int = 0,
) -> tuple[float, dict[int, float]]:
    j = 0
    wait = 0
    arrivals = 0
    time_include = 0.0
    occupancy_time: dict[int, float] = defaultdict(float)
    rng = random.Random(seed)

    for state in range(events):
        j_before = j
        if j == 0:
            arrival_interval = exp(lam, rng)
            service_interval = math.inf
        else:
            arrival_interval = exp(lam, rng)
            service_interval = exp(min(j, c) * mu, rng)

        is_arrival = arrival_interval <= service_interval
        interval = arrival_interval if is_arrival else service_interval

        if state >= warmup:
            time_include += interval
            occupancy_time[j_before] += interval

        if is_arrival:
            if state >= warmup:
                arrivals += 1
                if j_before >= c:
                    wait += 1
            j += 1
        else:
            j -= 1

    distribution = {
        n: value / time_include for n, value in sorted(occupancy_time.items())
    }
    wait_rate = wait / arrivals
    return wait_rate, distribution

```

```

def erlang_b(lam: float, mu: float, c: int) -> float:
    rho = lam / mu
    numerator = rho**c / math.factorial(c)
    denominator = sum(rho**k / math.factorial(k) for k in range(c + 1))
    return numerator / denominator

def erlang_c(lam: float, mu: float, c: int) -> float:
    rho = lam / mu
    if rho >= c:
        return math.nan
    a = rho**c / math.factorial(c)
    b = sum(rho**k / math.factorial(k) for k in range(c))
    return a / ((1.0 - rho / c) * b + a)

def mmcc_distribution_theory(lam: float, mu: float, c: int) -> np.ndarray:
    rho = lam / mu
    weights = np.array([rho**n / math.factorial(n) for n in range(c + 1)])
    return weights / weights.sum()

def mmc_distribution_theory(
    lam: float,
    mu: float,
    c: int,
    max_n: int,
) -> np.ndarray:
    rho = lam / mu
    normalizer = sum(rho**n / math.factorial(n) for n in range(c))
    normalizer += (rho**c / math.factorial(c)) * c / (c - rho)
    probs = []
    for n in range(max_n + 1):
        if n < c:
            weight = rho**n / math.factorial(n)
        else:
            weight = rho**n / (math.factorial(c) * c ** (n - c))
        probs.append(weight / normalizer)
    return np.array(probs)

def poisson_distribution(rho: float, max_n: int) -> np.ndarray:
    return np.array(
        [math.exp(-rho) * rho**n / math.factorial(n) for n in range(max_n + 1)]
    )

def distribution_to_array(distribution: dict[int, float], max_n: int) -> np.ndarray:
    return np.array([distribution.get(n, 0.0) for n in range(max_n + 1)])

def plot_waiting_probability() -> None:
    theory_x = np.linspace(0.01, 9.95, 400)
    sim_x = np.array(
        [0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0,
         5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.3, 9.6, 9.8]
    )
    sim_y = []

```

```

for i, lam in enumerate(sim_x):
    wait_rate, _ = simulation_mmc(
        lam, MU, C_TASK, seed=10_000 + i, events=EVENTS, warmup=WARMUP
    )
    sim_y.append(wait_rate)

plt.figure(figsize=(7, 4.6))
plt.plot(
    theory_x,
    [erlang_c(lam, MU, C_TASK) for lam in theory_x],
    label="Theory Erlang C",
    color="#1f77b4",
)
plt.scatter(sim_x, sim_y, label="Simulation", color="#d62728", s=28, zorder=3)
plt.xlabel(r"$E=\lambda/\mu$")
plt.ylabel("Waiting probability")
plt.xlim(0, 10)
plt.ylim(0, 1.02)
plt.grid(alpha=0.25)
plt.legend()
plt.tight_layout()
plt.savefig(OUT_DIR / "fig_waiting_probability.png", dpi=180)
plt.close()

def plot_blocking_probability() -> None:
    theory_x = np.linspace(0.01, 10.0, 400)
    sim_x = np.array(
        [0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0,
         5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.3, 9.6, 9.8]
    )
    sim_y = []
    for i, lam in enumerate(sim_x):
        block_rate, _ = simulation_mmcc(
            lam, MU, C_TASK, seed=20_000 + i, events=EVENTS, warmup=WARMUP
        )
        sim_y.append(block_rate)

    plt.figure(figsize=(7, 4.6))
    plt.plot(
        theory_x,
        [erlang_b(lam, MU, C_TASK) for lam in theory_x],
        label="Theory Erlang B",
        color="#1f77b4",
    )
    plt.scatter(sim_x, sim_y, label="Simulation", color="#d62728", s=28, zorder=3)
    plt.xlabel(r"$E=\lambda/\mu$")
    plt.ylabel("Blocking probability")
    plt.xlim(0, 10)
    plt.ylim(0, 0.25)
    plt.grid(alpha=0.25)
    plt.legend()
    plt.tight_layout()
    plt.savefig(OUT_DIR / "fig_blocking_probability.png", dpi=180)
    plt.close()

def plot_distribution_comparison() -> None:
    lam = 8.0
    mu = 1.0

```

```

rho = lam / mu
c_values = [10, 15, 20]
max_n = 35
n_axis = np.arange(max_n + 1)
poisson = poisson_distribution(rho, max_n)

fig, axes = plt.subplots(1, 2, figsize=(10, 4.4), sharey=True)
for ax, model_name in zip(axes, ["M/M/c", "M/M/c/c"], strict=True):
    ax.plot(n_axis, poisson, color="black", linewidth=2, label="Poisson(8)")
    for i, c in enumerate(c_values):
        seed = 30_000 + i if model_name == "M/M/c" else 40_000 + i
        if model_name == "M/M/c":
            _, sim_dist = simulation_mmc(
                lam, mu, c, seed=seed, events=EVENTS, warmup=WARMUP
            )
            theory = mmc_distribution_theory(lam, mu, c, max_n)
        else:
            _, sim_dist = simulation_mmcc(
                lam, mu, c, seed=seed, events=EVENTS, warmup=WARMUP
            )
            theory = np.zeros(max_n + 1)
            mmcc_theory = mmcc_distribution_theory(lam, mu, c)
            theory[: c + 1] = mmcc_theory

        sim = distribution_to_array(sim_dist, max_n)
        ax.plot(n_axis, theory, linewidth=1.5, label=f"Theory c={c}")
        ax.scatter(n_axis, sim, s=12, alpha=0.6, label=f"Simulation c={c}")

    ax.set_title(model_name)
    ax.set_xlabel("Number in system n")
    ax.set_xlim(0, max_n)
    ax.grid(alpha=0.25)

axes[0].set_ylabel("Probability")
axes[1].legend(fontsize=7, ncol=1)
fig.tight_layout()
fig.savefig(OUT_DIR / "fig_distribution_comparison.png", dpi=180)
plt.close(fig)

def main() -> None:
    plot_waiting_probability()
    plot_blocking_probability()
    plot_distribution_comparison()

if __name__ == "__main__":
    main()

```